Object Oriented Programming (Lab)

BSIT - Fall 2024

Dr. Muhammad Farooq

The objective of this lab is to:
To understand and utilize the concept of arrays (1D and 2D) along with the previous concepts.

ALERT!

1. This is an individual lab. You are **strictly NOT allowed to collaborate** with others, share screens, or communicate answers in any form.
2. Use of **AI tools (e.g., ChatGPT, Copilot, etc.) is strictly prohibited**. Any AI-generated content will be treated as academic dishonesty.
3. Anyone caught in the act of **cheating would be awarded a 0** in this Lab.
4. MAKE SURE TO **VALIDATE WHERE EVER YOU TAKE INPUT** FROM THE USER< OTHERWISE MARKS SHALL BE DEDUCTED.

Lab - 9

Task 01 (30 marks)

Implement the following class using multifiling.

```
class Complex
{
private:
    double real, imag;

public:
    // ========== CONSTRUCTORS ==========
    Complex(double r = 0.0, double i = 0.0);

    // ========== MEMBER OPERATOR OVERLOADING ==========
    Complex operator+(const Complex& rhs);          // Complex + Complex
    Complex& operator+(double rhs);                 // Complex + double
    Complex& operator+=(const Complex& rhs);        // Compound assignment
    Complex& operator=(const Complex& rhs);         // Assignment operator

    // ========== MEMBER FUNCTIONS ==========
    void display() const;
```

```
    double getReal() const;                          // Getters for
non-friend functions
    double getImag() const;

    // ========== FRIEND FUNCTION DECLARATIONS ==========
    friend Complex operator+(double lhs, const Complex& rhs);  // double +
Complex

    bool operator==(const Complex& rhs) const;       // Equality comparison
    bool operator!=(const Complex& rhs) const;       // Inequality
comparison
};

// ========== NON-MEMBER FUNCTION DECLARATIONS ==========
Complex operator+(double lhs, const Complex& rhs);   // Friend function
implementation

int main()
{
    Complex c1(3, 4);
    Complex c2(3, 4);

    // Testing various operator overloads
    Complex c3 = c1 + c2;        // Complex + Complex
    Complex c4 = c2 + 5.6;       // Complex + double
    Complex c5 = 5.6 + c2;       // double + Complex (friend function)

    // Cascading demonstration
    Complex c6 = c1 + c2 + c3;

    c3.display();
    c4.display();
    c5.display();

    // Relational operators
    if (c1 == c2) {
        cout << "c1 and c2 are equal" << endl;
    }

    return 0;
}
```

Task 02 (30 marks)

Implement the following class using multifiling.

```
class MyVector
{
private:
    int size;        // Current number of elements
    int capacity;    // Maximum capacity
    int* arr;        // Dynamic array

    void resize(int newCapacity); // Private helper for resizing

public:
    // ========== CONSTRUCTORS & DESTRUCTOR ==========
    MyVector();                     // Default capacity: 5
    MyVector(int capacity);         // Custom capacity
    MyVector(const MyVector& v);    // Copy constructor
    ~MyVector();                    // Destructor

    // ========== BASIC VECTOR OPERATIONS ==========
    void push_back(int element);
    void pop_back();
    void insert(int index, int element);
    void remove(int element);
    void clear();
    void reverse();
    int search(int element) const;  // Returns index or -1 if not found

    // ========== ACCESSORS & UTILITIES ==========
    int get_size() const;
    int get_capacity() const;
    void print() const;
    bool isEmpty() const;
    bool isMonotonicallyIncreasing() const;

    // ========== OPERATOR OVERLOADING ==========

    // Assignment Operator (Rule of Three)
    MyVector& operator=(const MyVector& rhs);

    // Arithmetic Operators (Member Functions)
```

```
    MyVector operator+(const MyVector& rhs) const;  // Vector addition
    MyVector& operator+=(const MyVector& rhs);      // Compound addition

    // Relational Operators (Member Functions)
    bool operator==(const MyVector& rhs) const;     // Equality check
    bool operator!=(const MyVector& rhs) const;     // Inequality check
comparison

   };
```